Production README & architecture guide: sortable API endpoints, auth flow diagrams, and deploy steps in one polished PDF

Computer
Architecture workflow & migration diff: map legacy product templates to new developer endpoints with exact code changes
Computer
Review the exact diff and file changes
Walk through the production migration plan
Inspect the integration test orchestrator
provide all nessary matrial inorder to launch deploy and atcvate entire platfrom with everything live i need entire eycossytm and platfrom built and launched

I created the polished production architecture PDF, a sortable API catalog, and a launch-control review pack containing the revised test orchestrator, exact proposed patch, migration walkthrough, template-to-endpoint map, and activation requirements.

The main finding is that “deploy everything” is not yet an executable release decision. At the inspected repository commit, the approved institutional adapter registry is empty, and the inspected startup does not supply the registration-readiness handler. Those are concrete gaps—not issues a README or deployment toggle can fix.

This delivery is **launch-control** material, not a claim that the entire ecosystem is built or activated. No repository, database, deployment, or institutional configuration was changed.

1. Materials delivered

File	What it provides
Production architecture and launch guide PDF	Developer setup, architecture flows, auth diagrams, API catalog, diff review, migration, onboarding, deployment, rollback, and activation gates
Sortable API catalog HTML	Filterable routes with sortable method, path, auth, purpose, and activation-gate columns
API catalog CSV	Spreadsheet-ready endpoint inventory
Launch-control review pack	Proposed patch, source-linked API records, reviewed orchestrator, tests, findings, and review manifest

The PDF’s tables are static. Sorting and filtering are provided in the accompanying HTML—not represented as an interactive PDF feature.

Verified in this delivery

- 14 gateway routes were cataloged from inspected source.
- 15 unit tests passed for the revised orchestrator.
- The proposed patch passed `git apply --check` in a clean temporary repository.
- All deliverables were verified as nonempty.

The application's browser tests, authenticated journeys, live endpoints, migration rehearsal, and rollback mechanisms were not executed here.

2. What the source inspection establishes

The guide uses this pinned baseline:

```
Repository: harrisonjrubin7-cmyk/semester
Commit:     4b4f8ca742242132f5b2dd254d23422ae70eaa7a
```

I inspected:

```
app/server/institution/start.ts
app/server/institution/gateway.ts
app/server/institution/auth.ts
app/server/institution/adapters.ts
```

The gateway return was truncated, so this is a targeted source review—not a full security or implementation audit.

Finding	Operational significance
`adapters.ts` exports an empty approved-adapter array	Adapter-dependent services cannot serve real institutional records through that registry
Startup adds sandbox adapters only under an explicit sandbox flag	Demonstration workflows remain separate from institutional approval
Registration-readiness handlers exist in the gateway	Route existence alone does not activate the service
Inspected startup omits the `registrationReadiness` dependency	Those handlers return unavailable under that startup configuration
Readiness can default to journal health	A successful health response can coexist with zero approved adapters
Institutional auth validates the token through the auth service and resolves membership	Personal sign-in does not automatically confer institutional access

These findings apply to the inspected source baseline. They do not establish the configuration of every currently running service.

The project catalogs likewise distinguish planned, prototype, implemented, and authoritative capabilities. Those distinctions must remain visible during completion and launch. [1] [2]

3. Source-defined API catalog

The catalog includes these observed gateway routes:

Area	Routes
Health	`GET /health/live`, `GET /health/ready`, `GET /health`
Authentication configuration	`GET /v1/auth/config`
Institutional status and records	`GET /status`, `GET /records`
Consequential actions	`POST /actions/prepare`, `/actions/commit`, `/actions/reconcile`
Intelligence	`GET /v1/intelligence/policy`, `POST /v1/intelligence/respond`, `POST /v1/intelligence/actions/:id/confirm`
Registration readiness	`POST /v1/registration-readiness/evaluations`, `POST /v1/registration-readiness/evaluations/:id/evaluate`

These are relative gateway paths—not verified public deployment URLs. They are also not a complete census of all Semester APIs.

Template-to-endpoint mapping

The guide distinguishes a supported mapping from an unverified one:

- Institution console → connection status and authorized records.
- Registrar action preview → prepare, commit, and reconcile.
- Registration-readiness screen → evaluation handlers, with startup wiring still required.
- AI response surface → policy, response, and bounded confirmation handlers.
- Course Studio → a domain workflow requiring verified authoring, section, submission, and feedback contracts; no generic course CRUD endpoint is invented.
- Privacy module → distinct preference, permission, consent, and session contracts; gateway auth is only one layer.
- Board portal → company reporting and disclosure contracts; no unverified financial or data-room API is presented as implemented.

A product screen route in the design catalog is not automatically an HTTP endpoint. [1]

4. Exact diff review

The previous proposed diff added documentation and test orchestration. It did not implement product capabilities or migrate the production database.

I identified and addressed several orchestrator weaknesses:

Finding	Revised behavior
Missing executable or OS launch errors were unhandled	Records an unavailable `launch_error` result
Setup failures could exit without a report	Attempts to persist a failed setup report
Local service key was forwarded to every child suite	Removes it except for account-sync
Captured child output could grow in memory	Discards child output by default
Loopback application URL could be mistaken for complete backend isolation	Explicitly requires inspection of build/backend configuration
Selected-suite success could be overinterpreted	Always reports `production_ready: false`

The revised runner still inherits other environment variables. It is not a process sandbox; use a clean ephemeral test environment.

Scope of the new patch

The review pack proposes a new documentation subtree containing:

- Launch requirements.
- Diff-review findings.
- Template-to-endpoint mapping.
- API catalog records.
- Revised orchestrator.
- Orchestrator tests.

It introduces no runtime gateway changes, approved adapters, database DDL, deployment configuration, or feature activation.

The earlier generated archive was unavailable in this execution environment. The review used the exact source shown in the prior conversation; the revised package is a new additive proposal, not an unverified binary modification of that archive. During repository integration, choose one canonical runner rather than maintaining competing launch mechanisms.

5. Production migration walkthrough

A production migration must start with the real target and schema—not the earlier SQLite demonstration.

Required sequence

1. Resolve the approved target
   Confirm the database project, application, functions, region, and sole migration executor.
2. Compare histories and actual objects
   Reconcile repository migration hashes, remote ledger, tables, constraints, functions, grants, and policies.

3. Design the expand change
   Specify the exact capability requirement and backward-compatible schema/API additions.
4. Rehearse
   Test disposable PostgreSQL with synthetic existing-state fixtures, old/new application compatibility, tenant denials, retries, lock impact, and recovery.
5. Approve execution
   Bind approval to the exact SQL, target, hash, owner, backup evidence, and abort conditions.
6. Apply through one executor
   Verify the resulting ledger and objects—not only a dashboard status.
7. Deploy compatible services and activate the approved domain
   Observe actual role workflows, receipts, and reconciliation.
8. Contract later
   Remove obsolete structures only after incompatible rollback candidates are retired.

No production DDL is included because the exact required schema delta and target mapping remain unverified. Application rollback does not reverse schema changes or recover lost data.

6. What is necessary to activate the entire ecosystem

Keep the full scope, but require evidence for every capability.

```
Domain
  → capability and accountable owner
  → implemented workflow
  → permitted roles and denied access
  → screens and all relevant states
  → data ownership and authority
  → source and freshness
  → execution, receipt, and reconciliation
  → positive and negative tests
  → deployment and configuration fingerprint
  → institutional approval where required
  → operational support and rollback
  → activation receipt
```

The domain catalog already separates learning, registration, advising, finance, campus, career, family, governance, integrations, company operations, and shared foundations. Completing one domain does not establish readiness of the others. [2]

Required work packages

Work package	Owner role	Completion evidence
Identity and workspace boundaries	Identity/security owner	Membership, revocation, and cross-context denial tests
Course Studio and assessment	Learning owner	Authoring through submission, review, release, and recovery

Work package	Owner role	Completion evidence
Registration and academic records	Registrar/domain owner	Current-source validation, approval, uncertain-outcome reconciliation
Privacy and devices	Privacy owner	Real permissions, consent, expiry, sessions, export/deletion evidence
Advising and services	Domain owners	Scoped case workflows, referrals, escalation, continuity
Finance and commerce	Finance systems owner	Reconciled records, provider boundaries, authorization
Company and board reporting	Finance/management	Reconciled metrics and approved audience disclosures
Shared search and AI	Data/AI owners	Permission-aware retrieval, provider approval, bounded-action evaluations
Production operations	Release/operations owner	Monitoring, restore, rollback, incident response
Institutional activation	Institutional sponsor	Approved scope, configuration, authority, and acceptance

The project's authority gates explicitly require proof before native learning or academic-record systems become official. A deployment cannot bypass those gates.[2]

7. Deployment and rollback

The guide maps this release sequence:

```
Verified implementation
 → PR checks and registry evidence
 → isolated preview and migration rehearsal
 → immutable release manifest
 → blocking release gate and approval
 → compatible database expansion
 → functions/API release
 → application release
 → observed role journeys
 → domain-specific activation
```

Rollback is separate:

```
Authorized containment
 → tested compatible prior version
 → schema compatibility check
 → rollback of affected delivery surface
 → actual page, cache, auth, and write verification
 → reconciliation of in-flight work
 → incident record
```

Frontend, API/functions, database, and domain enablement are independent surfaces. One rollback does not control them all.

The next implementation step is to turn the verified blockers into reviewed runtime changes: resolve production environment ownership, complete the relevant service wiring, implement or approve real domain adapters where needed, and execute the missing domain tests. Until that work has evidence, labeling the entire platform “live” would hide the exact gaps the launch process needs to resolve.

⁂

1. The Main Semester design system (2) copy/docs/native-platform/NATIVE_SCREEN_CATALOG.md
2. The Main Semester design system (2) copy/docs/native-platform/NATIVE_DOMAIN_CATALOG.md